Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.1.3 sur 153 | difficulté 0 / 10

0.1.3. La fonction poser() — poser une tuile

#include <poser> : poser(colonne, ligne, tuile) pose une seule tuile — une lettre, un dessin — dans une case.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `poser(colonne, ligne, tuile)` pose **une tuile** dans une case : une lettre de la police, un dessin à toi, ou un numéro.

La ligne à écrire : `#include <poser>`**.** Une seule ligne : ici, la tuile se donne par son **numéro**. Les lettres de la police sont rangées dans l'ordre : la tuile **1** est le A, la **2** le B… la **26** le Z. L'étape suivante, `ALPHABET`, donne un nom à ces numéros.

Ses arguments : la colonne, la ligne, la tuile. Les trois peuvent se calculer.

Ce qu'elle coûte : environ **120 octets** la première fois (l'écran mis en place, et la lettre), puis une dizaine par appel.

Essaie : pose un B (la tuile 2) dans chaque coin de l'écran.

On retrouve poser dans 192 leçons

0.2, 0.2.1, 0.2.2, 0.5, 0.5.1, 0.5.2, 0.6, 0.6.1, 0.6.2, 0.7, 0.7.1, 0.7.2, 0.8, 0.8.1, 0.8.2, 0.9, 0.9.1, 0.9.2, 0.12, 0.12.1, 0.12.2, 0.13, 0.13.1, 0.13.2, 0.14, 0.14.1, 0.14.2, 0.15, 0.15.1, 0.15.2, 0.16, 0.16.1, 0.16.2, 0.17, 0.17.1, 0.17.2, 0.30, 0.30.1, 0.30.2, 0.31, 0.31.1, 0.31.2, 0.33, 0.33.1, 0.33.2, 0.65, 0.65.1, 0.65.2, 0.66, 0.66.1, 0.66.2, 0.67, 0.67.1, 0.67.2, 0.68, 0.68.1, 0.68.2, 0.69, 0.69.1, 0.69.2, 0.76.1, 0.76.2, 0.76.3, 0.76.4, 0.76.5, 0.76.6, 0.76.8, 0.77, 0.78, 0.78.1, 0.81, 0.81.1, 0.81.3, 0.82, 0.82.1, 0.83, 0.83.2, 0.85, 0.91, 0.91.1, 0.91.2, 0.91.3, 0.91.4, 0.91.5, 0.91.6, 0.91.7, 0.91.8, 0.91.9, 0.92, 0.92.1, 0.92.2, 0.93, 0.93.1, 0.93.2, 0.93.3, 0.93.4, 0.93.5, 0.93.6, 0.93.7, 0.93.8, 0.93.9, 0.94, 0.94.1, 0.95, 0.95.1, 0.96, 0.96.1, 0.97, 0.97.1, 0.98, 0.98.1, 0.99, 0.99.1, 0.100, 0.100.1, 0.101, 0.101.1, 0.102, 0.102.1, 0.103, 0.104, 0.105, 0.105.1, 0.106, 0.107, 0.107.1, 0.108, 3, 13, 34, 35, 36, 37, 38, 39, 40, 41, 42, 44, 47, 52, 60, 61, 62, 63, 64, 72, 73, 74, 76, 84, 85, 95, 96, 97, 98, 99, 101, 102, 103, 105, 106, 110.3, 110.13, 110.14, 110.15, 110.16, 110.18, 110.20, 112, 113, 113.4, 122, 124, 125, 132, 133, 134, 135, 136, 137, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153.5, 153.9

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <poser> : poser une tuile ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <poser> : poser une tuile ---- »). La console l'ignore.
`#include <poser>  // poser() : une tuile dans une case`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « poser() : une tuile dans une case ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`poser(9, 8, 1);  // la tuile numéro 1 : le A, au milieu`	Appel de la fonction poser de la console, avec les arguments 9, 8, 1. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la tuile numéro 1 : le A, au milieu ».)

`poser(0, 0, 26);  // la tuile numéro 26 : le Z, en haut à gauche`	Appel de la fonction poser de la console, avec les arguments 0, 0, 26. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la tuile numéro 26 : le Z, en haut à gauche ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **206 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           19 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  CopierTuiles     40 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles           32 o  ← CopierTuiles  — 2 lettres sur 43
Lettres gravées : AZ
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 2    12 o sur place, dans main()
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <poser> : poser une tuile ----

#include <poser>      // poser() : une tuile dans une case
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

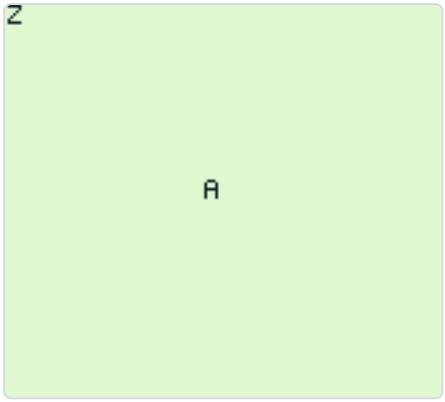
```
int main() {
  poser(9, 8, 1);      // la tuile numéro 1 : le A, au milieu
  poser(0, 0, 26);     // la tuile numéro 26 : le Z, en haut à gauche

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> Un A au milieu, un Z en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A est en (9, 8)

✓ le Z est en (0, 0)

Ensuite : **0.1.4. ALPHABET — les lettres de la police** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++